

NỀN TẢNG AI TẠO SINH
(IT5410 – Foundation of Generative AI)

SCORE-BASED MODELS

Phạm Minh Hiếu
Thân Quang Khoát
Trường CNTT&TT, ĐHBKHN

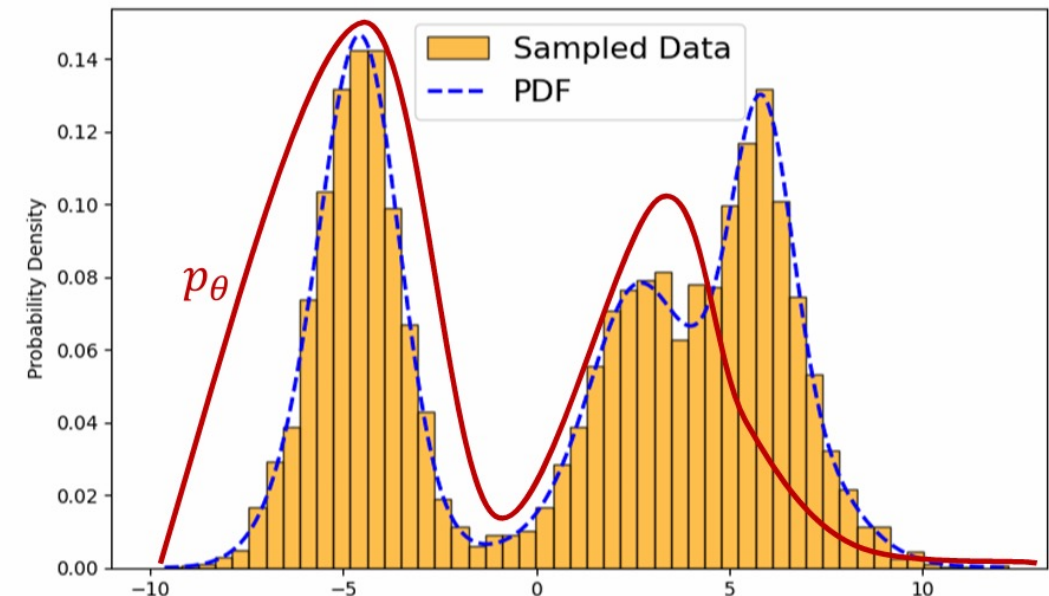
2024

Nội dung

- Mở đầu
- Một số vấn đề của Học sâu
- Một số kiến trúc mạng nơron
- **Mô hình sinh sâu**
- Đánh giá chất lượng
- Học tăng cường

Learning a generative model

- Given a training set of examples, e.g., images of dogs
- We want to learn a probability distribution $P(\mathbf{x})$ over image $\mathbf{x}$ such that
 - **Generation:** If we sample $\mathbf{x}_{new} \sim P(\mathbf{x})$, $\mathbf{x}_{new}$ should look like a dog (sampling)
 - **Density estimation:** $P(\mathbf{x})$
 - **Unsupervised representation learning:** We should be able to learn what these images have in common, e.g., ears, tail, ...
- In practice
 - We often find a $P_{\theta}(\mathbf{x})$ to **approximate** $P(\mathbf{x})$ as “**close**” as possible
 - How do we evaluate “closeness”?
- How to choose a good model family?
 - $P_{\theta}(\mathbf{x})$



Score estimation: Minimizing the Fisher divergence

- **Score function:** $s(\mathbf{x}) = \nabla_{\mathbf{x}} \log p(\mathbf{x})$
- **Fisher divergence:** Given two distribution p and q whose pdfs are differentiable, the *Fisher divergence* between p and q is given by:

$$D_F(p, q) = \frac{1}{2} \mathbf{E}_{\mathbf{x} \sim p(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log p(\mathbf{x}) - \nabla_{\mathbf{x}} \log q(\mathbf{x})\|_2^2]$$

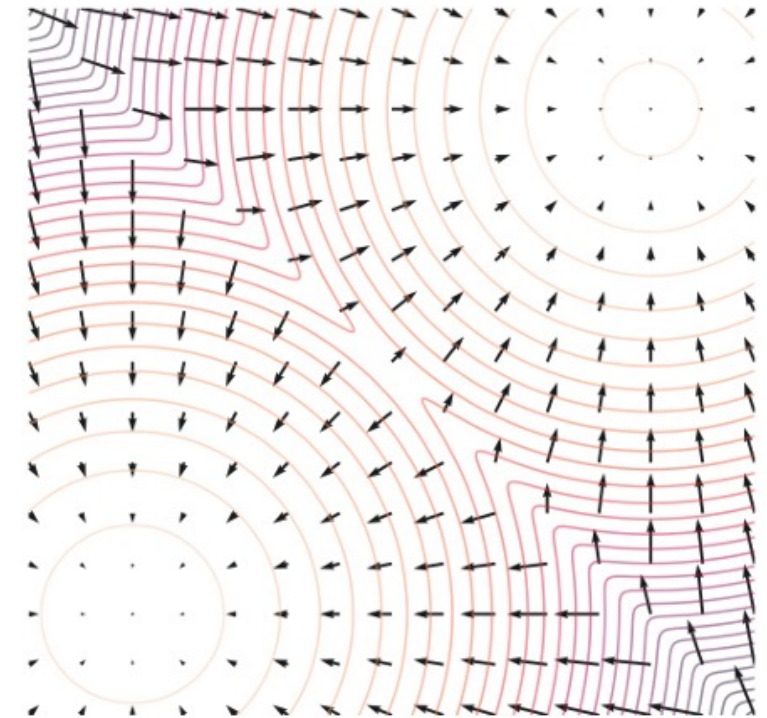
- **Score matching:** minimizing the Fisher divergence between $P(\mathbf{x})$ and $p_{\theta}(\mathbf{x})$:

$$\min_{\theta} \frac{1}{2} \mathbf{E}_{\mathbf{x} \sim P(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log P(\mathbf{x}) - s_{\theta}(\mathbf{x})\|_2^2]$$

where $s_{\theta}(\mathbf{x}): \mathbb{R}^d \rightarrow \mathbb{R}^d$ is the *score model*.

(d is the number of dimensions)

- **Problem:** $P(\mathbf{x})$ is unknown!



Visualization of pdf and score function of a distribution

Hyvarinen A., Estimation of Non-Normalized Statistical Models by Score Matching, JMLR (2005)

Score matching: Minimizing the Fisher divergence

- Fortunately, we can directly work with the objective after some mathematical transformations (using integration by parts)!

$$\min_{\theta} \frac{1}{2} \mathbf{E}_{\mathbf{x} \sim P(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log P(\mathbf{x}) - s_{\theta}(\mathbf{x})\|_2^2]$$

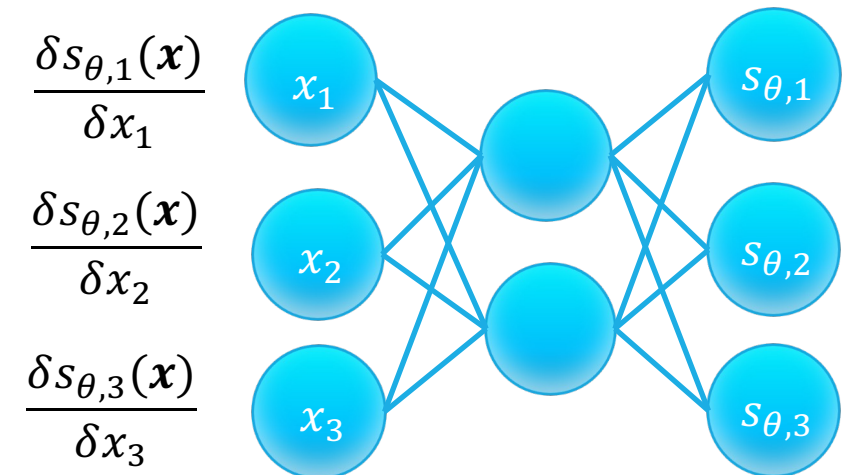
- The objective is equivalent to:

$$\min_{\theta} \mathbf{E}_{\mathbf{x} \sim P(\mathbf{x})} \left[\frac{1}{2} \|s_{\theta}(\mathbf{x})\|_2^2 + \text{tr}(\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x})) \right]$$

Jacobian of
score function

- **Problem:** Computing $\text{tr}(\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}))$ requires $O(d)$ backprops.

→ Not scalable as the number of dimensions increases!



Score matching: Training procedure

- Sample a minibatch of data points $B = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\} \sim P(\mathbf{x})$.
- Estimate the score matching loss and optimize:

$$L_{SM}(B, \theta) = \frac{1}{m} \sum_{i=1}^m \left(\frac{1}{2} \|s_{\theta}(\mathbf{x}_i)\|_2^2 + \text{tr}(\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}_i)) \right)$$

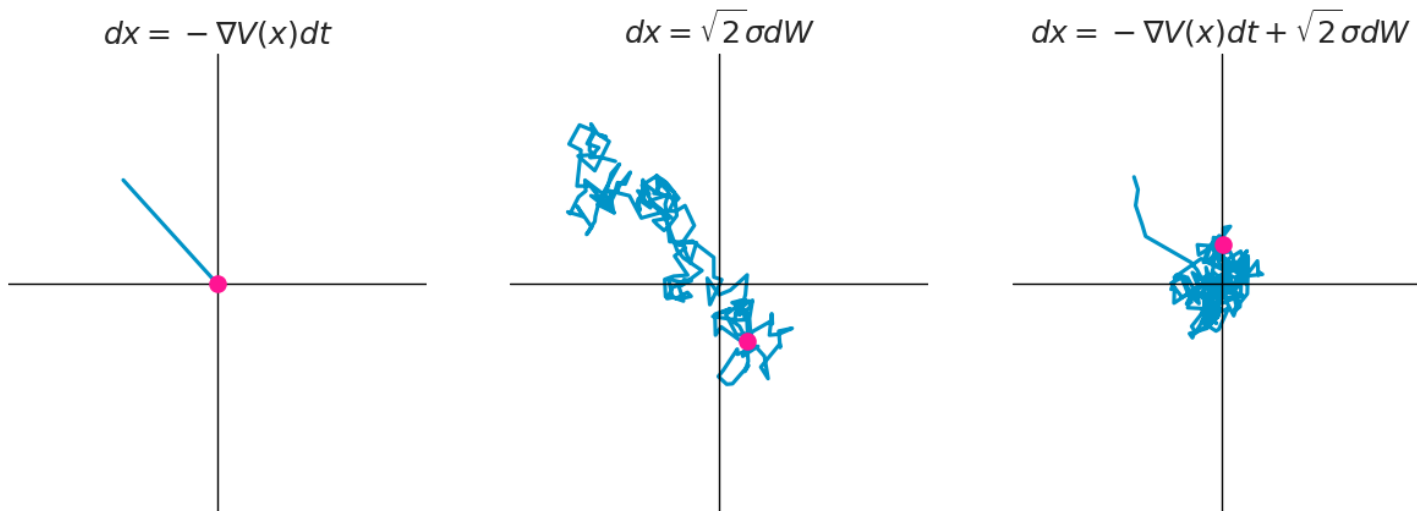
- Repeat until convergence.

Score matching: Sampling via Langevin dynamics

- Allow sampling given only the score function of a distribution.
- The Langevin dynamics for sampling from a distribution $p(\mathbf{x})$ is given by the following stochastic differential equation (SDE):

$$d\mathbf{x} = \underbrace{\nabla \log p(\mathbf{x}) dt}_{\text{drift term}} + \underbrace{\sqrt{2} \sigma d\mathbf{w}}_{\text{volatility}}$$

where $\mathbf{w}$ denotes the standard Brownian motion (Wiener process).



Source: [Langevin Monte Carlo: Sampling using Langevin Dynamics](#)

Score matching: Sampling via Langevin dynamics

Langevin Markov Chain Monte Carlo:

- Sample $\mathbf{x}_0 \sim \pi(\mathbf{x})$, where $\pi(\mathbf{x})$ is a prior distribution that is easy to sample from.

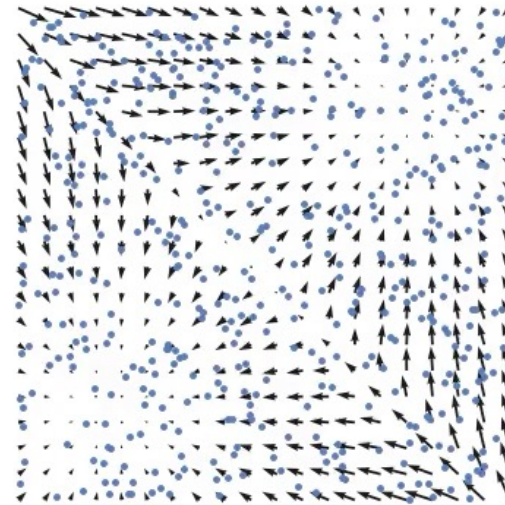
- For $t = 0, 1, \dots, T - 1$:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \epsilon \nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x}) + \sqrt{2\epsilon} \mathbf{z}_t,$$

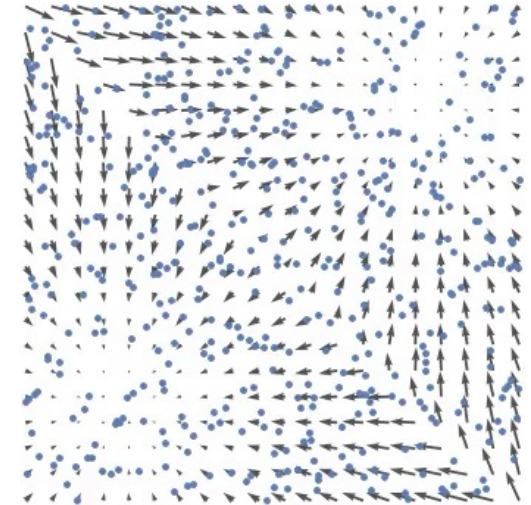
where $\mathbf{z}_t \sim N(0, \mathbf{I})$.

- After training our score-based model:
 $\nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x}) \approx s_{\theta}(\mathbf{x})$.

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \epsilon s_{\theta}(\mathbf{x}) + \sqrt{2\epsilon} \mathbf{z}_t$$



Follow the score only
 $\mathbf{x}_{t+1} = \mathbf{x}_t + \epsilon s_{\theta}(\mathbf{x})$



Langevin MCMC

Source: CS236 – Stanford University

Denoising score matching

- Working with the noise-perturbed distribution instead of the original one.

$$q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) = N(\mathbf{x}; \mathbf{0}, \sigma^2 \mathbf{I})$$
$$q_{\sigma}(\tilde{\mathbf{x}}) = \int P(\mathbf{x}) q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) d\mathbf{x}$$

- Score matching on $q_{\sigma}(\tilde{\mathbf{x}})$:

$$\min_{\theta} \frac{1}{2} \mathbf{E}_{\tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}})} [\|\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}}) - s_{\theta}(\tilde{\mathbf{x}})\|_2^2]$$

- Again, using mathematical transformations, we can convert the objective to a tractable one:

$$\min_{\theta} \frac{1}{2} \mathbf{E}_{\mathbf{x} \sim P(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x})} [\|s_{\theta}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x})\|_2^2]$$

Denoising score matching

Since $q_\sigma(\tilde{\mathbf{x}} | \mathbf{x}) = N(\mathbf{x}; \mathbf{0}, \sigma^2 \mathbf{I})$, the density function of $q_\sigma(\tilde{\mathbf{x}} | \mathbf{x})$ is given by:

$$q_\sigma(\tilde{\mathbf{x}} | \mathbf{x}) = \frac{1}{\sqrt{\det(2\pi\sigma^2 \mathbf{I})}} \exp\left(\frac{-1}{2\sigma^2} \|\tilde{\mathbf{x}} - \mathbf{x}\|_2^2\right)$$

Taking the natural logarithm of both sides:

$$\log q_\sigma(\tilde{\mathbf{x}} | \mathbf{x}) = \log \frac{1}{\sqrt{\det(2\pi\sigma^2 \mathbf{I})}} - \frac{1}{2\sigma^2} \|\tilde{\mathbf{x}} - \mathbf{x}\|_2^2$$

Hence:

$$\nabla_{\tilde{\mathbf{x}}} \log q_\sigma(\tilde{\mathbf{x}} | \mathbf{x}) = \frac{-(\tilde{\mathbf{x}} - \mathbf{x})}{\sigma^2}$$

Denoising score matching

The objective

$$\min_{\theta} \frac{1}{2} \mathbf{E}_{\mathbf{x} \sim P(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x})} [\|s_{\theta}(\tilde{\mathbf{x}}) - \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x})\|_2^2]$$

can be written as:

$$\min_{\theta} \frac{1}{2} \mathbf{E}_{\mathbf{x} \sim P(\mathbf{x}), \tilde{\mathbf{x}} \sim q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x})} \left[\left\| s_{\theta}(\tilde{\mathbf{x}}) + \frac{\tilde{\mathbf{x}} - \mathbf{x}}{\sigma^2} \right\|_2^2 \right]$$

Denoising score matching: Training procedure

- Sample a minibatch of data points $B = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\} \sim P(\mathbf{x})$.
- Sample a minibatch of perturbed data points $\tilde{B} = \{\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2, \dots, \tilde{\mathbf{x}}_m\} \sim q_\sigma(\tilde{\mathbf{x}} | \mathbf{x})$.
- Estimate the denoising score matching loss and optimize:

$$L_{DSM}(B, \tilde{B}, \theta) = \frac{1}{2m} \sum_{i=1}^m \left\| s_\theta(\tilde{\mathbf{x}}_i) + \frac{\tilde{\mathbf{x}}_i - \mathbf{x}_i}{\sigma^2} \right\|_2^2$$

- Repeat until convergence.

Denoising score matching: Sampling

- We can obtain samples from $q_\sigma(\tilde{\mathbf{x}})$ via Langevin dynamics.
- Given a noisy sample, how to get a ‘clean’ sample?
- Two natural approaches:
 - Choose the ‘most likely’ clean sample.

$$\mathbf{x}^* = \arg \max_{\mathbf{x}} P(\mathbf{x} \mid \tilde{\mathbf{x}})$$



Intractable!

- Choose the expected clean sample.

$$\mathbf{x}^* = \mathbf{E}_{\mathbf{x} \sim P(\mathbf{x} \mid \tilde{\mathbf{x}})}[\mathbf{x}]$$

Here, $p(\mathbf{x} \mid \tilde{\mathbf{x}})$ is given by Bayes' formula:

$$P(\mathbf{x} \mid \tilde{\mathbf{x}}) = \frac{q_\sigma(\tilde{\mathbf{x}} \mid \mathbf{x})P(\mathbf{x})}{q_\sigma(\tilde{\mathbf{x}})}$$

Denoising score matching: Sampling

- We consider the second approach (choosing the expected clean sample).

$$\begin{aligned}
 E_{x \sim P(x | \tilde{x})}[\mathbf{x}] &= \tilde{\mathbf{x}} + E_{x \sim P(x | \tilde{x})}[\mathbf{x} - \tilde{\mathbf{x}}] \\
 &= \tilde{\mathbf{x}} + \int P(\mathbf{x} | \tilde{\mathbf{x}})(\mathbf{x} - \tilde{\mathbf{x}}) d\mathbf{x} \\
 &= \tilde{\mathbf{x}} + \int \frac{q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) P(\mathbf{x})}{q_{\sigma}(\tilde{\mathbf{x}})} \sigma^2 \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) d\mathbf{x} \\
 &= \tilde{\mathbf{x}} + \sigma^2 \int \frac{P(\mathbf{x})}{q_{\sigma}(\tilde{\mathbf{x}})} \nabla_{\tilde{\mathbf{x}}} q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) d\mathbf{x} \\
 &= \tilde{\mathbf{x}} + \frac{\sigma^2}{q_{\sigma}(\tilde{\mathbf{x}})} \nabla_{\tilde{\mathbf{x}}} \int P(\mathbf{x}) q_{\sigma}(\tilde{\mathbf{x}} | \mathbf{x}) d\mathbf{x} \\
 &= \tilde{\mathbf{x}} + \frac{\sigma^2}{q_{\sigma}(\tilde{\mathbf{x}})} \nabla_{\tilde{\mathbf{x}}} q_{\sigma}(\tilde{\mathbf{x}}) \\
 &= \tilde{\mathbf{x}} + \sigma^2 \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}})
 \end{aligned}$$

Denoising score matching: Sampling

- Tweedie's formula:

$$E_{x \sim P(x | \tilde{x})}[\mathbf{x}] = \tilde{\mathbf{x}} + \sigma^2 \nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}})$$

- After training our score-based model: $\nabla_{\tilde{\mathbf{x}}} \log p_{\theta}(\tilde{\mathbf{x}}) \approx s_{\theta}(\tilde{\mathbf{x}})$.

$$E_{x \sim P(x | \tilde{x})}[\mathbf{x}] \approx \tilde{\mathbf{x}} + \sigma^2 s_{\theta}(\tilde{\mathbf{x}})$$

- A suitable denoising strategy: follow the score (gradient)!

Sliced score matching

- Instead of working with the original score, we consider projections onto random directions.
- One dimensional problems should be much easier.
- **Objective:** minimizing the *sliced Fisher divergence*.

$$\min_{\theta} \frac{1}{2} \mathbf{E}_{\mathbf{v} \sim p_{\mathbf{v}}} \mathbf{E}_{\mathbf{x} \sim P(\mathbf{x})} \left[\left(\mathbf{v}^T \nabla_{\mathbf{x}} \log p_{data}(\mathbf{x}) - \mathbf{v}^T s_{\theta}(\mathbf{x}) \right)^2 \right]$$

- Again (for the third time!), we can transform the objective into a tractable one:

$$\min_{\theta} \mathbf{E}_{\mathbf{v} \sim p_{\mathbf{v}}} \mathbf{E}_{\mathbf{x} \sim P(\mathbf{x})} \left[\mathbf{v}^T \nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}) \mathbf{v} + \frac{1}{2} \left(\mathbf{v}^T s_{\theta}(\mathbf{x}) \right)^2 \right]$$

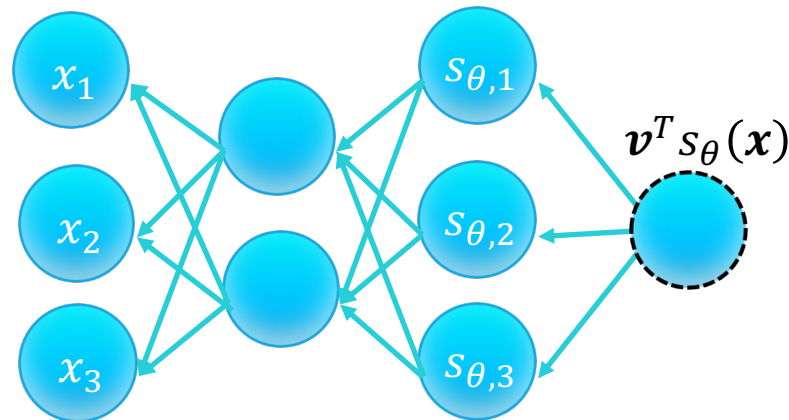
- The objective involves $\nabla_{\mathbf{x}} s_{\theta}(\mathbf{x})$, which seems unscalable.

Sliced score matching

- A trick to reduce complexity:

$$\begin{array}{ccc} \mathbf{v}^T \nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}) & \mathbf{v} = \mathbf{v}^T \nabla_{\mathbf{x}} (s_{\theta}(\mathbf{x})) \\ \text{\textit{O}(d) backprops} & \text{\textit{O}(1) backprop!} \end{array}$$

→ Sliced score matching is scalable!



Sliced score matching: Training procedure

- Sample a minibatch of data points $B = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\} \sim P(\mathbf{x})$.
- Sample a minibatch of projection directions $V = \{\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_m\} \sim p_v$. The projection distribution is usually standard Gaussian or multivariate Rademacher.
- Estimate the sliced score matching loss and optimize:

$$L_{SSM}(B, V, \theta) = \frac{1}{m} \sum_{i=1}^m \left[\mathbf{v}_i^T \nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}_i) \mathbf{v}_i + \frac{1}{2} \left(\mathbf{v}_i^T s_{\theta}(\mathbf{x}_i) \right)^2 \right]$$

- Repeat until convergence.

- Recall the training objective:

$$\min_{\theta} \mathbf{E}_{\mathbf{v} \sim p_{\mathbf{v}}} \mathbf{E}_{\mathbf{x} \sim P(\mathbf{x})} \left[\mathbf{v}^T \nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}) \mathbf{v} + \frac{1}{2} (\mathbf{v}^T s_{\theta}(\mathbf{x}))^2 \right]$$

- Denote $\mathbf{v} = [v_1, v_2, \dots, v_d]^T$ and $s_{\theta}(\mathbf{x}) = [s_{\theta}(\mathbf{x})_1, s_{\theta}(\mathbf{x})_2, \dots, s_{\theta}(\mathbf{x})_d]^T$.
- If the projection distribution is Standard Gaussian or multivariate Rademacher, then

$$\begin{aligned} \mathbf{E}_{\mathbf{v} \sim p_{\mathbf{v}}} \left[(\mathbf{v}^T s_{\theta}(\mathbf{x}))^2 \right] &= \mathbf{E}_{\mathbf{v} \sim p_{\mathbf{v}}} \left[\left(\sum_{i=1}^d v_i s_{\theta}(\mathbf{x})_i \right)^2 \right] \\ &= \mathbf{E}_{\mathbf{v} \sim p_{\mathbf{v}}} \left[\sum_{i=1}^d v_i^2 s_{\theta}(\mathbf{x})_i^2 + \sum_{1 \leq i \neq j \leq d} v_i v_j s_{\theta}(\mathbf{x})_i s_{\theta}(\mathbf{x})_j \right] \\ &= \sum_{i=1}^d s_{\theta}(\mathbf{x})_i^2 \underbrace{\mathbf{E}_{v_i \sim p(v_i)} [v_i^2]}_1 + \sum_{1 \leq i \neq j \leq d} s_{\theta}(\mathbf{x})_i s_{\theta}(\mathbf{x})_j \underbrace{\mathbf{E}_{v_i, v_j \sim p(v_i, v_j)} [v_i v_j]}_0 \\ &= \|s_{\theta}(\mathbf{x})\|_2^2 \end{aligned}$$

- The above transformations apply for all projection distributions of mean $\mathbf{0}$ and covariance $\mathbf{I}$.

- The objective can be rewritten as:

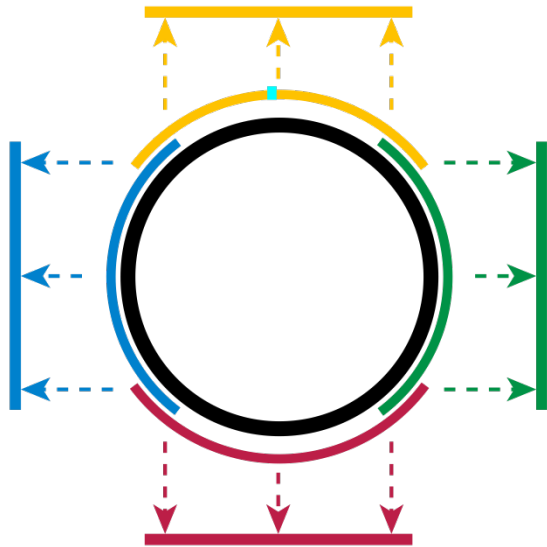
$$\min_{\theta} E_{\mathbf{x} \sim P(\mathbf{x})} \left[E_{\mathbf{v} \sim p_{\mathbf{v}}} [\mathbf{v}^T \nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}) \mathbf{v}] + \frac{1}{2} \|s_{\theta}(\mathbf{x})\|_2^2 \right]$$

- We can train using an estimator with **reduced variance**:

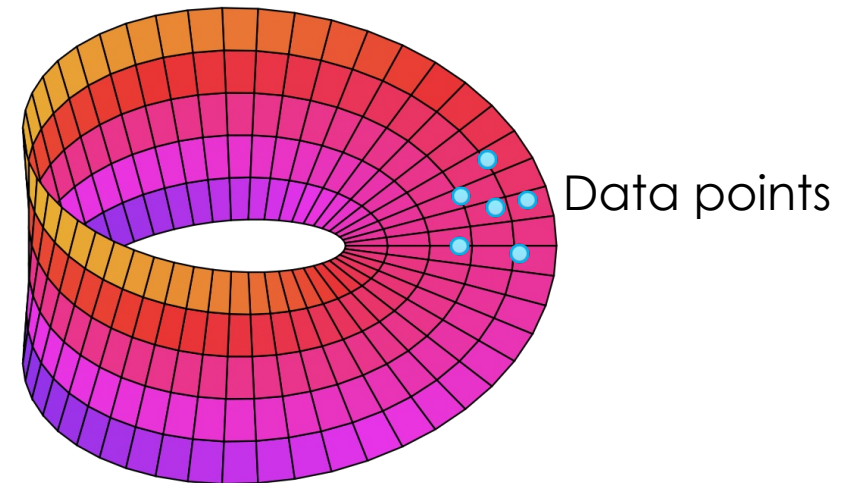
$$L_{SSM_RV}(B, \theta) = \frac{1}{m} \sum_{i=1}^m \left[\mathbf{v}_i^T \nabla_{\mathbf{x}} s_{\theta}(\mathbf{x}_i) \mathbf{v}_i + \frac{1}{2} \|s_{\theta}(\mathbf{x}_i)\|_2^2 \right]$$

Score matching: pitfalls

- For non-differentiable $\log p(\mathbf{x})$, the score is not defined!
- **Manifold hypothesis:** high-dimensional datasets actually lie along low-dimensional latent manifolds.



A circle is a 1-D manifold.
(Source: [Manifold \[Wikipedia\]](#))



Data points lie in low-dimensional manifolds
(Source: CS236, Stanford University)

Score matching: pitfalls

- Score matching fails in low data density regions.

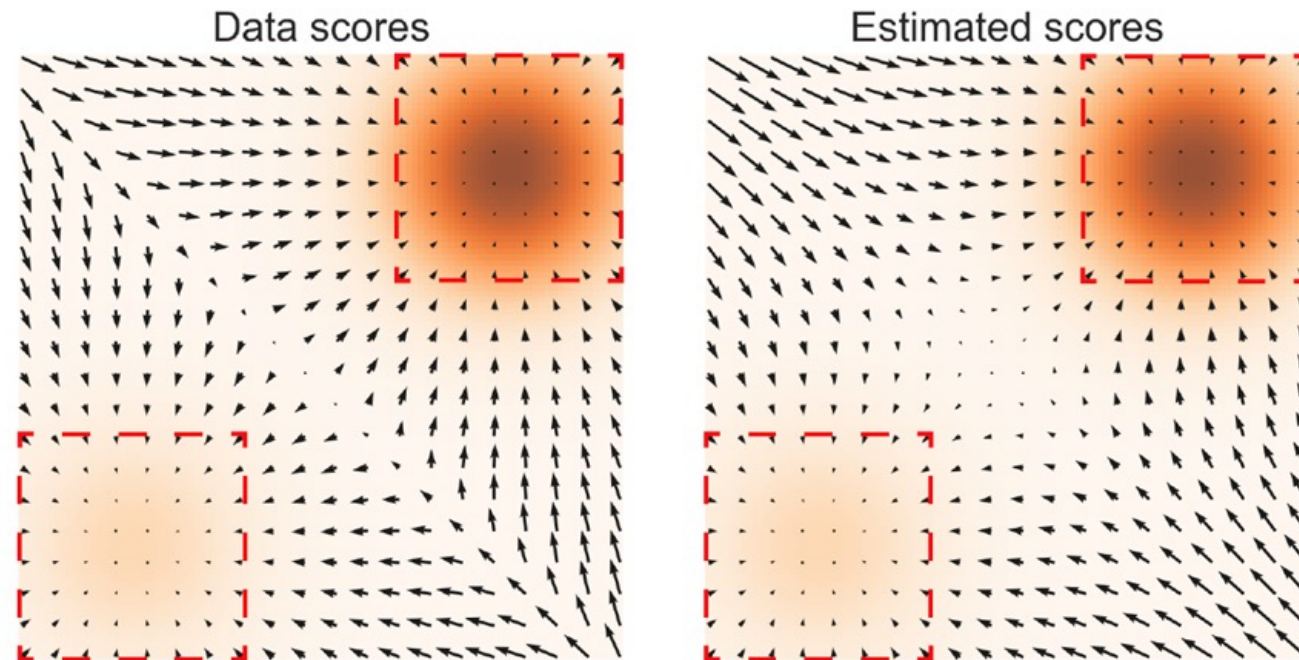
- Recall the training objective of score matching:

$$\begin{aligned} & \frac{1}{2} \mathbf{E}_{\mathbf{x} \sim P(\mathbf{x})} [\|\nabla_{\mathbf{x}} \log P(\mathbf{x}) - s_{\theta}(\mathbf{x})\|_2^2] \\ &= \frac{1}{2} \int P(\mathbf{x}) \|\nabla_{\mathbf{x}} \log P(\mathbf{x}) - s_{\theta}(\mathbf{x})\|_2^2 d\mathbf{x} \end{aligned}$$

- In regions where no data points are sampled ($P(\mathbf{x}) \approx 0$), score matching is inaccurate.

Score matching: pitfalls

- Score matching fails in low data density regions.
- Derailment when sampling by Langevin MCMC.

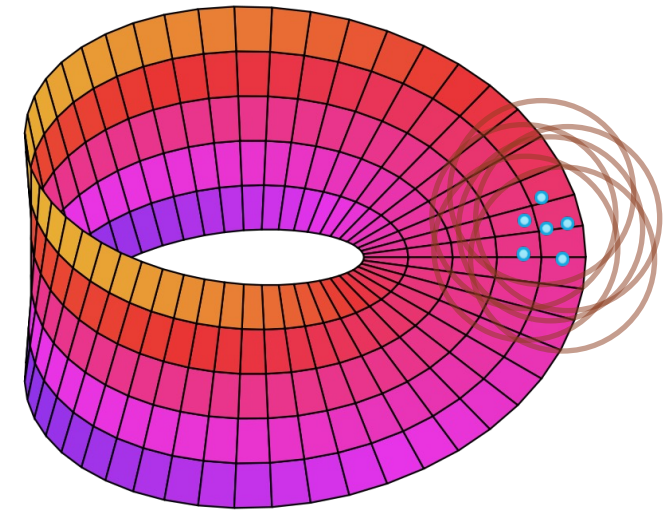


Using sliced score matching to estimate scores of a mixture of Gaussians

Denoising score matching rises!

- Working on a Gaussian-perturbed distribution can solve all of the problems mentioned above.
- **Theorem:** *If X is a random variable and ϵ is an absolutely continuous random variable, then $X + \epsilon$ is an absolutely continuous random variable. As a result, the density of $X + \epsilon$ is differentiable almost everywhere.*

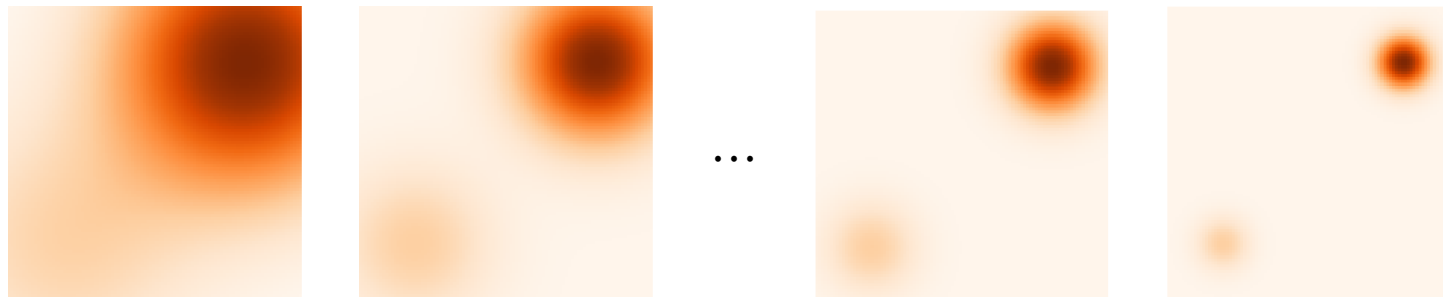
→ Solve the problem of undefined score function!



Source: CS236 – Stanford University

- Working on a Gaussian-perturbed distribution can solve all of the problems mentioned above.
- But how much noise is enough?
 - High noise: covers more low density region but might over-corrupt the data.
 - Low noise: less corruption of the original data distribution, but reduces estimation accuracy.
- **Solution: use multiple scales of noise perturbations.**

$$\sigma_1 > \sigma_2 > \cdots > \sigma_{L-1} > \sigma_L$$



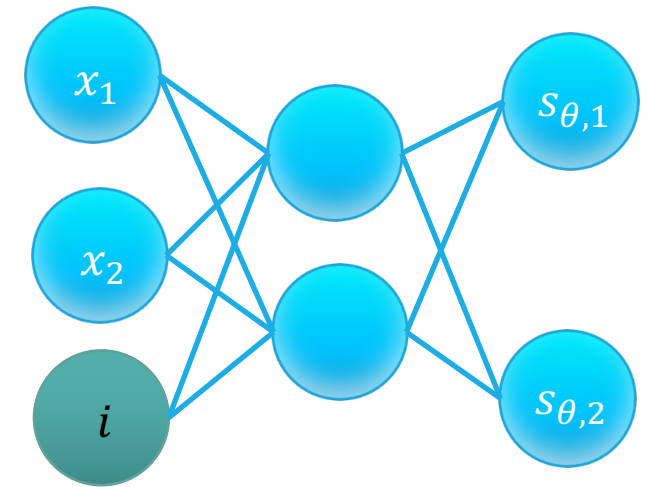
- Suppose there are L decreasing standard deviations $\sigma_1 > \sigma_2 > \dots > \sigma_L$. For each Gaussian noise $N(\mathbf{0}, \sigma_i^2 \mathbf{I})$, its corresponding noise-perturbed distribution is given by:

$$p_{\sigma_i}(\tilde{\mathbf{x}}) = \int P(\mathbf{x}) N(\tilde{\mathbf{x}}; \mathbf{x}, \sigma_i^2 \mathbf{I}) d\mathbf{x}$$

- We want to train a **Noise Conditional Score Network (NCSN)** $s_\theta(\tilde{\mathbf{x}}, i)$, such that:

$$s_\theta(\tilde{\mathbf{x}}, i) \approx \nabla_{\tilde{\mathbf{x}}} \log p_{\sigma_i}(\mathbf{x})$$

- How to train and design a NCSN?



A noise conditional score network.

NCSN: objective

- For each σ_i , the denoising score matching objective is given by:

$$l(\theta, \sigma_i) = \frac{1}{2} \mathbf{E}_{x \sim P(x)} \mathbf{E}_{\tilde{x} \sim N(x, \sigma_i^2 I)} \left[\left\| s_{\theta}(\tilde{x}, i) + \frac{\tilde{x} - x}{\sigma_i^2} \right\|_2^2 \right]$$

- The objective of NCSN is a weighted sum of denoising score matching losses:

$$L(\theta, \{\sigma_i\}_{i=1}^L) = \frac{1}{L} \sum_{i=1}^L \lambda(\sigma_i) l(\theta, \sigma_i),$$

where $\lambda(\sigma_i) > 0$ is the weighting function.

- In practice, the weighting function is often chosen as $\lambda(\sigma_i) = \sigma_i^2$.

NCSN: training procedure

- Sample a minibatch of data points $B = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\} \sim P(\mathbf{x})$.
- Sample a minibatch of indices $T = \{i_1, i_2, \dots, i_m\} \sim U\{1, 2, \dots, L\}$, where U denotes the uniform distribution.
- Sample a minibatch of noise-perturbed samples $\tilde{B} = \{\tilde{\mathbf{x}}_1, \tilde{\mathbf{x}}_2, \dots, \tilde{\mathbf{x}}_m\}$, where $\tilde{\mathbf{x}}_k \sim N(\mathbf{x}_k, \sigma_{i_k}^2 \mathbf{I})$.

- Estimate the loss and optimize with respect to θ :

$$L(B, T, \tilde{B}, \theta) = \frac{1}{2m} \sum_{k=1}^m \lambda(\sigma_{i_k}) \left\| s_{\theta}(\tilde{\mathbf{x}}_k, i_k) + \frac{\tilde{\mathbf{x}}_k - \mathbf{x}_k}{\sigma_{i_k}^2} \right\|_2^2$$

- Repeat until convergence.

Annealed Langevin dynamics:

- Sampling by running Langevin dynamics for $i = 1, 2, \dots, L - 1, L$ in sequence.
- Sample from p_{σ_i} is used as initialization for sampling from $p_{\sigma_{i+1}}$.

Algorithm 1 Annealed Langevin dynamics.

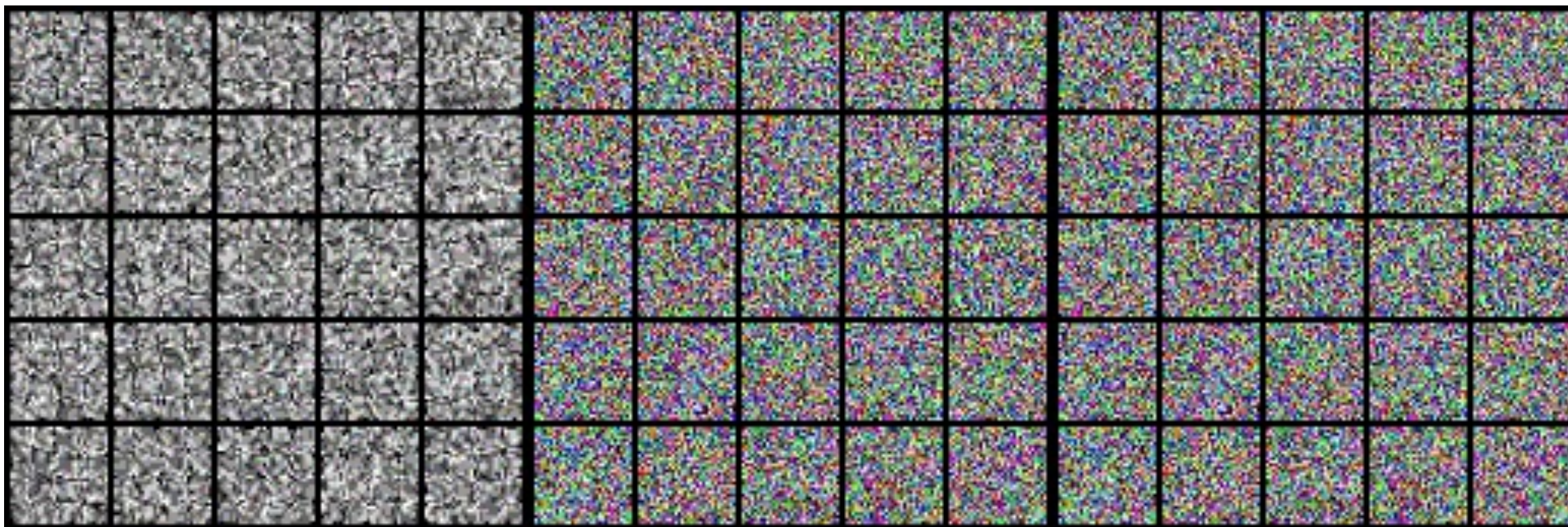
Require: $\{\sigma_i\}_{i=1}^L, \epsilon, T$.

- 1: Initialize $\tilde{\mathbf{x}}_0$
 - 2: **for** $i \leftarrow 1$ to L **do**
 - 3: $\alpha_i \leftarrow \epsilon \cdot \sigma_i^2 / \sigma_L^2$ $\triangleright \alpha_i$ is the step size.
 - 4: **for** $t \leftarrow 1$ to T **do**
 - 5: Draw $\mathbf{z}_t \sim \mathcal{N}(0, I)$
 - 6: $\tilde{\mathbf{x}}_t \leftarrow \tilde{\mathbf{x}}_{t-1} + \frac{\alpha_i}{2} \mathbf{s}_\theta(\tilde{\mathbf{x}}_{t-1}, \sigma_i) + \sqrt{\alpha_i} \mathbf{z}_t$
 - 7: **end for**
 - 8: $\tilde{\mathbf{x}}_0 \leftarrow \tilde{\mathbf{x}}_T$
 - 9: **end for**
- return $\tilde{\mathbf{x}}_T$
-

NCSN: choosing noise scales

Some recommendation from the authors:

- Choose $\{\sigma_i\}_{i=1}^L$ as a decreasing geometric progression. This ensures that the samples from high-density regions of $p_{\sigma_i}(\mathbf{x})$ are also likely to be in the high-density regions of $p_{\sigma_{i+1}}(\mathbf{x})$.
- Choose σ_1 as large as the maximum Euclidean distance between all pairs of training data points.



Model	Inception	FID
CIFAR-10 Unconditional		
PixelCNN [59]	4.60	65.93
PixelIQN [42]	5.29	49.46
EBM [12]	6.02	40.58
WGAN-GP [18]	$7.86 \pm .07$	36.4
MoLM [45]	$7.90 \pm .10$	18.9
SNGAN [36]	$8.22 \pm .05$	21.7
ProgressiveGAN [25]	$8.80 \pm .05$	-
NCSN (Ours)	$8.87 \pm .12$	25.32
CIFAR-10 Conditional		
EBM [12]	8.30	37.9
SNGAN [36]	$8.60 \pm .08$	25.5
BigGAN [6]	9.22	14.73

Table 1: Inception and FID scores for CIFAR-10

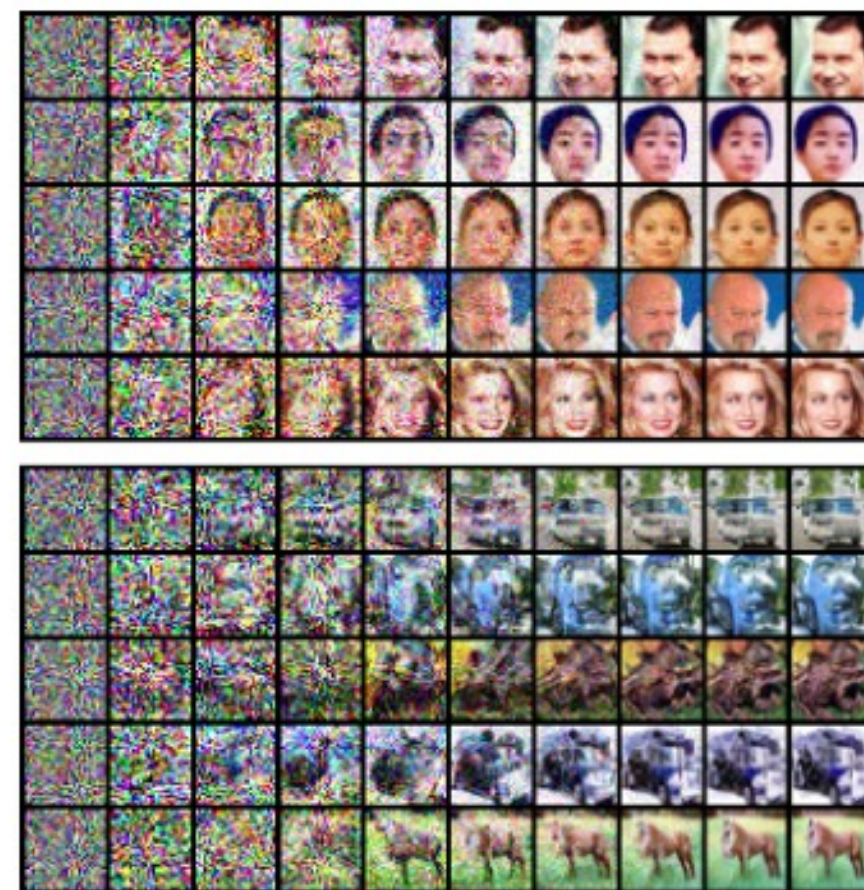


Figure 4: Intermediate samples of annealed Langevin dynamics.

Perturbing data with a diffusion process

- Let $\{\mathbf{x}(t) \in R^d\}_{t=0}^T$ be a diffusion process, indexed by the continuous time variable $t \in [0, T]$ and corresponds with a SDE of the form:

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w},$$

where:

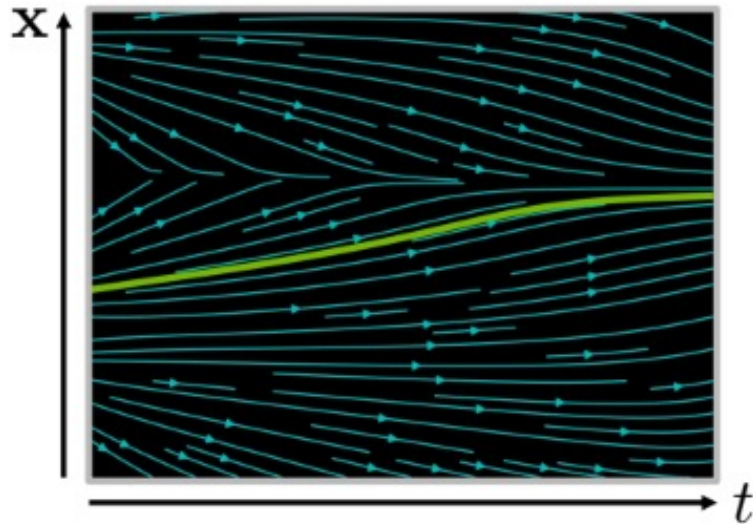
- $\mathbf{f}(\cdot, t): R^d \rightarrow R^d$: drift term
- $g(t) \in R$: diffusion term
- $\mathbf{w}$: standard Brownian motion
- Denote $p_t(\mathbf{x})$ be the probability distribution of $\mathbf{x}(t)$. We consider diffusion process such that:
 - $p_0 = P$
 - $p_T \approx \pi$, where π is a prior distribution that is easy to sample from.

Perturbing data with a diffusion process

34

Ordinary Differential Equation (ODE):

$$\frac{dx}{dt} = f(x, t) \quad \text{or} \quad dx = f(x, t)dt$$

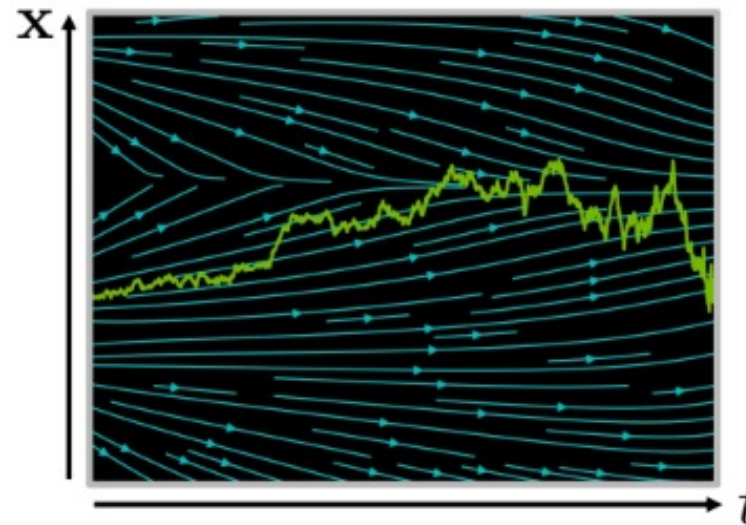


Analytical Solution: $x(t) = x(0) + \int_0^t f(x, \tau) d\tau$

Iterative Numerical Solution: $x(t + \Delta t) \approx x(t) + f(x(t), t)\Delta t$

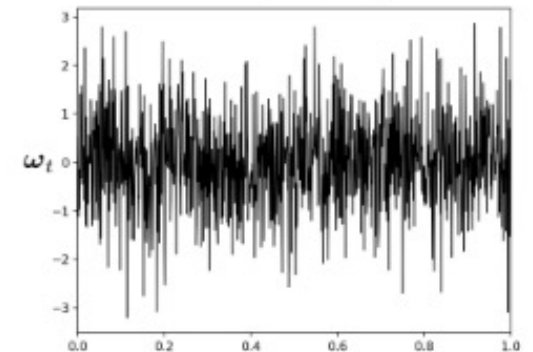
Stochastic Differential Equation (SDE):

$$\frac{dx}{dt} = \underbrace{f(x, t)}_{\text{drift coefficient}} + \underbrace{\sigma(x, t)\omega_t}_{\text{diffusion coefficient}}$$
$$\left(dx = f(x, t)dt + \sigma(x, t)d\omega_t \right)$$



$$x(t + \Delta t) \approx x(t) + f(x(t), t)\Delta t + \sigma(x(t), t)\sqrt{\Delta t}\mathcal{N}(\mathbf{0}, \mathbf{I})$$

Wiener Process
(Gaussian
White Noise)

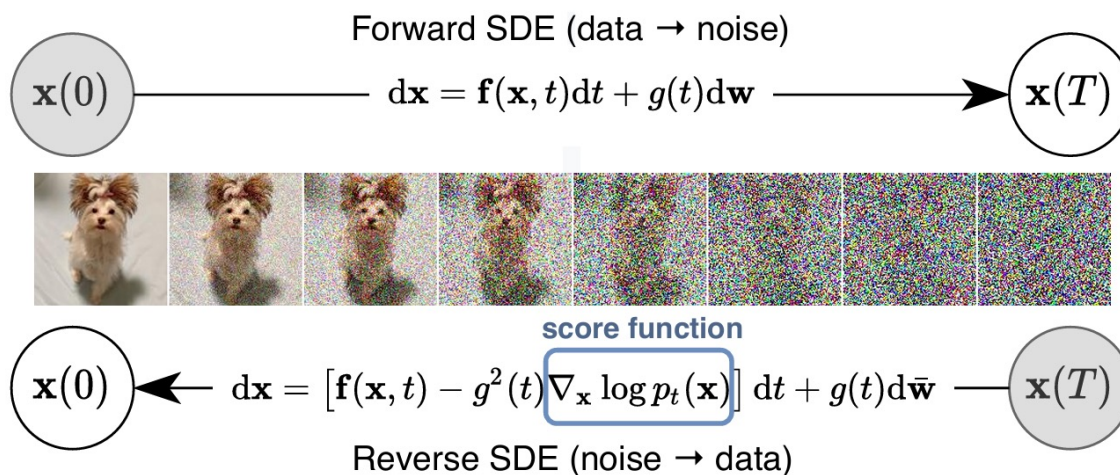


Reversing the diffusion process

- By starting from a sample from the prior distribution $\pi \approx p_T$ and reversing the diffusion process, we will be able to obtain a sample from p_0 .
- The reverse process is also a diffusion process running backwards in time, given by the following SDE:

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x})]dt + g(t)d\bar{\mathbf{w}},$$

where $\bar{\mathbf{w}}$ is a Brownian motion in the reverse time direction, and dt represents an infinitesimal negative time step.



Score-based model with SDE: objective

- Train a time-dependent score-based model $s_\theta(\mathbf{x}, t)$ such that:

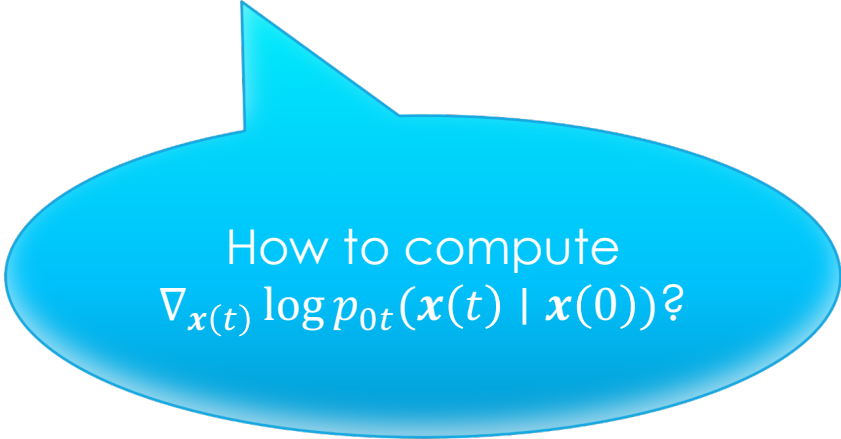
$$s_\theta(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$$

- **Objective:** weighted sum of denoising score matching objectives:

$$\min_{\theta} \mathbf{E}_{t \sim U(0, T)} \left[\lambda(t) \mathbf{E}_{\mathbf{x}(0) \sim p_0(\mathbf{x})} \mathbf{E}_{\mathbf{x}(t) \sim p_{0t}(\mathbf{x}(t) | \mathbf{x}(0))} \left[\left\| s_\theta(\mathbf{x}(t), t) - \nabla_{\mathbf{x}(t)} \log p_{0t}(\mathbf{x}(t) | \mathbf{x}(0)) \right\|_2^2 \right] \right],$$

where:

- $p_{0t}(\mathbf{x}(t) | \mathbf{x}(0))$: transition distribution from $p_0(\mathbf{x})$ to $p_t(\mathbf{x})$.
- $U(0, T)$: uniform distribution over $[0, T]$
- $\lambda(t) > 0$: weighting function.
 - A good choice is $\lambda(t) = g^2(t)$ (we'll discuss about this choice in later slides)



How to compute
 $\nabla_{\mathbf{x}(t)} \log p_{0t}(\mathbf{x}(t) | \mathbf{x}(0))$?

Designing the diffusion process

- Consider a diffusion process $\{\mathbf{x}(t) \in \mathbb{R}^d\}_{t=0}^T$ given by the following SDE:

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

We need to design $\mathbf{f}$ and g such that $p_0 = P$ and $p_T \approx \pi$, where π is a prior distribution that is easy to sample from.

- Some ways of designing the SDE:

- Variance Exploding (VE) SDE

$$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}} d\mathbf{w}$$

- Variance Preserving (VP) SDE

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}dt + \sqrt{\beta(t)}d\mathbf{w}$$

- Sub-VP SDE

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}dt + \sqrt{\beta(t)(1 - e^{-2 \int_0^t \beta(s)ds})} d\mathbf{w}$$

$$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}} d\mathbf{w}$$

- Euler-Maruyama (discretizing the above SDE): for $\Delta t \approx 0$ and $\mathbf{z}(t) \sim N(\mathbf{0}, \mathbf{I})$.

$$\begin{aligned} \mathbf{x}(t + \Delta t) &\approx \mathbf{x}(t) + \sqrt{\frac{\sigma^2(t + \Delta t) - \sigma^2(t)}{\Delta t}} \cdot (\mathbf{w}(t + \Delta t) - \mathbf{w}(t)) \\ &\approx \mathbf{x}(t) + \sqrt{\frac{\sigma^2(t + \Delta t) - \sigma^2(t)}{\Delta t}} \cdot \sqrt{\Delta t} \mathbf{z}(t) \\ &\approx \mathbf{x}(t) + \sqrt{\sigma^2(t + \Delta t) - \sigma^2(t)} \mathbf{z}(t) \end{aligned}$$

- Discretized version: $\mathbf{x}_{i+1} = \mathbf{x}_i + \sqrt{\sigma_{i+1}^2 - \sigma_i^2} \mathbf{z}_i$, where $\mathbf{z}_i \sim N(\mathbf{0}, \mathbf{I})$ and $\sigma_0 = 0$.
- By induction, one can show that: $p(\mathbf{x}_i | \mathbf{x}_0) = N(\mathbf{x}_0, \sigma_i^2 \mathbf{I})$ for all i .

→ Similar to score-based models with multiple noise perturbations!

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x}dt + \sqrt{\beta(t)}d\mathbf{w}$$

- Euler-Maruyama (discretizing the above SDE): for $\Delta t \approx 0$ and $\mathbf{z}(t) \sim N(\mathbf{0}, \mathbf{I})$:

$$\begin{aligned}\mathbf{x}(t + \Delta t) &\approx \mathbf{x}(t) - \frac{1}{2}\beta(t)\mathbf{x}(t)\Delta t + \sqrt{\beta(t)}(\mathbf{w}(t + \Delta t) - \mathbf{w}(t)) \\ &\approx \left(1 - \frac{1}{2}\beta(t)\Delta t\right) \mathbf{x}(t) + \sqrt{\beta(t)\Delta t} \mathbf{z}(t)\end{aligned}$$

- By Taylor expansion: $1 - \frac{1}{2}\beta(t)\Delta t \approx \sqrt{1 - \beta(t)\Delta t}$. Hence:

$$\begin{aligned}\mathbf{x}(t + \Delta t) &\approx \sqrt{1 - \beta(t)\Delta t} \mathbf{x}(t) + \sqrt{\beta(t)\Delta t} \mathbf{z}(t) \\ &\approx \sqrt{1 - \bar{\beta}(t)} \mathbf{x}(t) + \sqrt{\bar{\beta}(t)} \mathbf{z}(t),\end{aligned}$$

where we denote $\bar{\beta}(t) = \beta(t)\Delta t$.

- Discretized version: $\mathbf{x}_{i+1} = \sqrt{1 - \bar{\beta}_i} \mathbf{x}_i + \sqrt{\bar{\beta}_i} \mathbf{z}_i$, where $\mathbf{z}_i \sim N(\mathbf{0}, \mathbf{I})$.

→ Similar to Denoising Diffusion Probabilistic Models (DDPM)!

Designing the diffusion process

For VE, VP and sub-VP SDEs, their corresponding transition distributions $p_{0t}(\mathbf{x}(t) \mid \mathbf{x}(0))$ are all Gaussian and given as follows:

- For VE SDE:

$$p_{0t}(\mathbf{x}(t) \mid \mathbf{x}(0)) = N(\mathbf{x}(t); \mathbf{x}(0), [\sigma^2(t) - \sigma^2(0)]\mathbf{I})$$

- For VP SDE:

$$p_{0t}(\mathbf{x}(t) \mid \mathbf{x}(0)) = N\left(\mathbf{x}(t); \mathbf{x}(0)e^{-\frac{1}{2}\int_0^t \beta(s)ds}, \left[1 - e^{-\int_0^t \beta(s)ds}\right]\mathbf{I}\right)$$

- For sub-VP SDE:

$$p_{0t}(\mathbf{x}(t) \mid \mathbf{x}(0)) = N\left(\mathbf{x}(t); \mathbf{x}(0)e^{-\frac{1}{2}\int_0^t \beta(s)ds}, \left[1 - e^{-\int_0^t \beta(s)ds}\right]^2 \mathbf{I}\right)$$

Score-based model with SDE: Training procedure

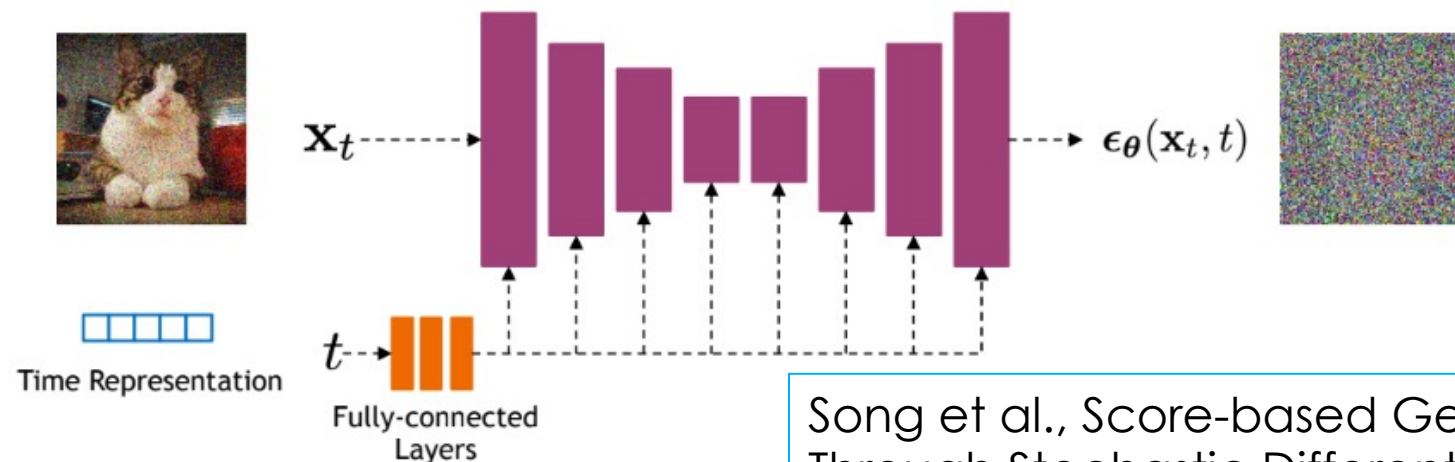
- Sample a minibatch of data points $B = \{\mathbf{x}_1(0), \mathbf{x}_2(0), \dots, \mathbf{x}_m(0)\} \sim p_0(\mathbf{x})$.
- Sample a minibatch of time index $T = \{t_1, t_2, \dots, t_m\} \sim U(0, T)$.
- Sample $\tilde{B} = \{\mathbf{x}_1(t_1), \mathbf{x}_2(t_2), \dots, \mathbf{x}_m(t_m)\}$, where $\mathbf{x}_i(t_i) \sim p_{0t_i}(\mathbf{x}(t_i) \mid \mathbf{x}_i(0))$
- Estimate the loss and optimize with respect to θ :

$$L_{SM_SDE}(B, T, \tilde{B}, \theta) = \frac{1}{m} \sum_{i=1}^m \lambda(t_i) \|s_{\theta}(\mathbf{x}_i(t_i), t_i) - \nabla_{\mathbf{x}(t_i)} \log p_{0t_i}(\mathbf{x}_i(t_i) \mid \mathbf{x}_i(0))\|$$

- Repeat until convergence.

Score-based model with NCSN & SDE: Network architecture

- Quite similar to the network architecture of DDPM: **U-Net architectures with ResNet blocks and self-attention layers**, with some additional improvements.
Check out the original paper (cited in the bottom right corner) for more details!
- Time representation: using Gaussian random features.
 - Sample $\omega \sim N(\mathbf{0}, s^2 \mathbf{I})$ and freeze ω (ω is not learnable).
 - For a time step t , the corresponding Gaussian random feature is defined as the vector:
$$[\sin(2\pi\omega t); \cos(2\pi\omega t)]$$



Score-based model with SDE: Sampling

- Recall the SDE of the reverse diffusion process:

$$d\mathbf{x} = [\mathbf{f}(\mathbf{x}, t) - g^2(t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x})]dt + g(t)d\bar{\mathbf{w}}$$

- **Sampling with Numerical SDE Solvers:**

- Sample from the prior: $\mathbf{x}(T) \sim \pi$

- Iteratively solve for $\mathbf{x}(0)$ by Euler-Maruyama approach: for $\Delta t \approx 0$:

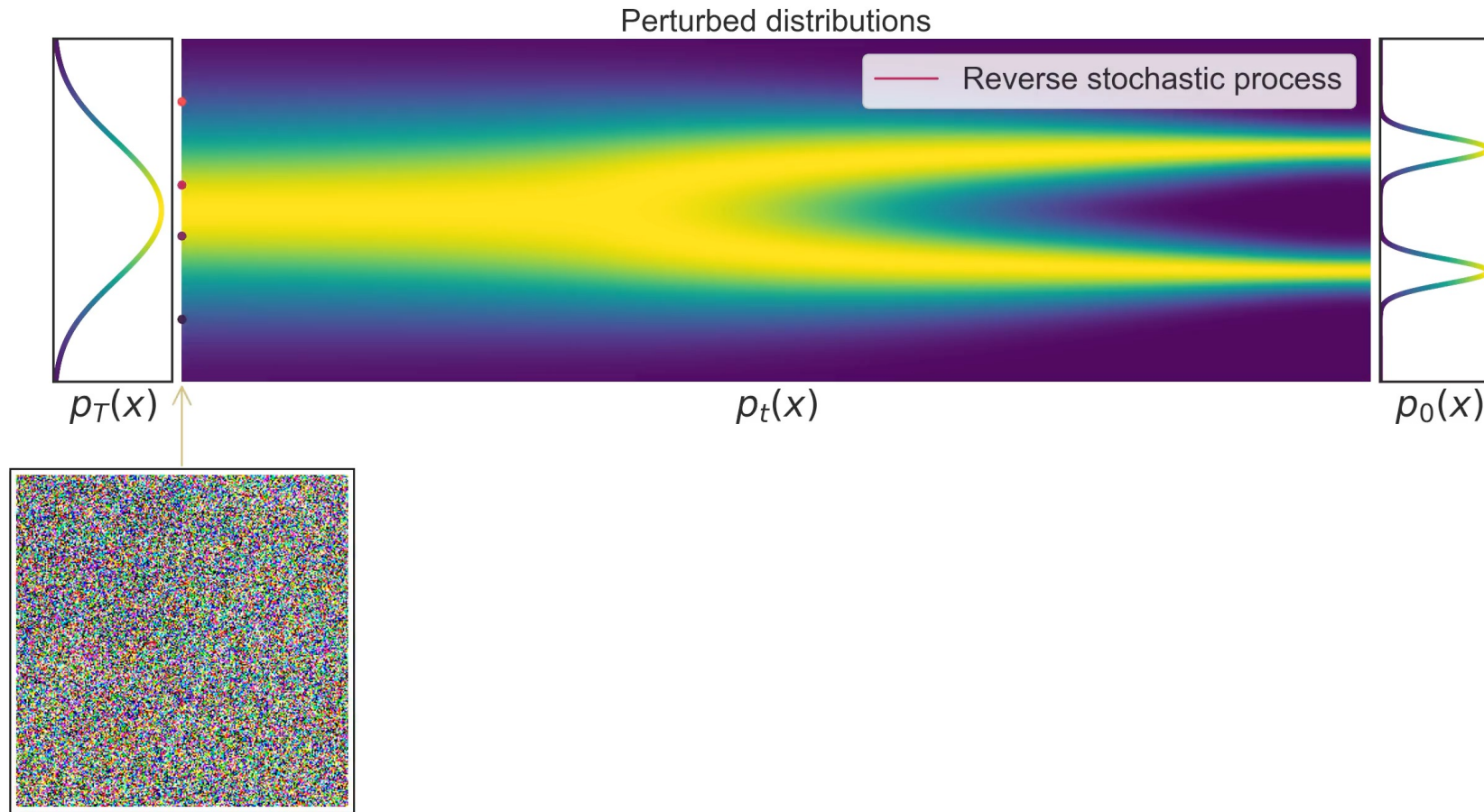
$$\begin{aligned}\mathbf{x}_{t-\Delta t} &= \mathbf{x}_t - [\mathbf{f}(\mathbf{x}_t, t) - g^2(t)s_{\theta}(\mathbf{x}_t, t)]\Delta t + g(t)\sqrt{\Delta t}\mathbf{z}_t; \\ t &\leftarrow t - \Delta t;\end{aligned}$$

where $\mathbf{z}_t \sim N(\mathbf{0}, \mathbf{I})$.

- Δt controls the trade-off between number of sampling steps and image quality.

Score-based model with SDE: Sampling

44



Score-based model with SDE: Sampling

Sampling with Predictor-Corrector method

- Apply one step of Numerical SDE solver to obtain $\mathbf{x}_{t-\Delta t}$ from $\mathbf{x}_t$ (predictor step)
- Apply several steps of Langevin MCMC to refine $\mathbf{x}_{t-\Delta t}$, such that $\mathbf{x}_{t-\Delta t}$ becomes a more accurate (higher density) sample from $p_{t-\Delta t}(\mathbf{x})$ (corrector step)

Algorithm 2 PC sampling (VE SDE)

```

1:  $\mathbf{x}_N \sim \mathcal{N}(\mathbf{0}, \sigma_{\max}^2 \mathbf{I})$ 
2: for  $i = N - 1$  to  $0$  do
3:    $\mathbf{x}'_i \leftarrow \mathbf{x}_{i+1} + (\sigma_{i+1}^2 - \sigma_i^2) \mathbf{s}_{\theta*}(\mathbf{x}_{i+1}, \sigma_{i+1})$ 
4:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:    $\mathbf{x}_i \leftarrow \mathbf{x}'_i + \sqrt{\sigma_{i+1}^2 - \sigma_i^2} \mathbf{z}$ 
6:   for  $j = 1$  to  $M$  do
7:      $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
8:      $\mathbf{x}_i \leftarrow \mathbf{x}_i + \epsilon_i \mathbf{s}_{\theta*}(\mathbf{x}_i, \sigma_i) + \sqrt{2\epsilon_i} \mathbf{z}$ 
9: return  $\mathbf{x}_0$ 

```

Algorithm 3 PC sampling (VP SDE)

```

1:  $\mathbf{x}_N \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $i = N - 1$  to  $0$  do
3:    $\mathbf{x}'_i \leftarrow (2 - \sqrt{1 - \beta_{i+1}}) \mathbf{x}_{i+1} + \beta_{i+1} \mathbf{s}_{\theta*}(\mathbf{x}_{i+1}, i + 1)$ 
4:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:    $\mathbf{x}_i \leftarrow \mathbf{x}'_i + \sqrt{\beta_{i+1}} \mathbf{z}$  Predictor
6:   for  $j = 1$  to  $M$  do Corrector
7:      $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
8:      $\mathbf{x}_i \leftarrow \mathbf{x}_i + \epsilon_i \mathbf{s}_{\theta*}(\mathbf{x}_i, i) + \sqrt{2\epsilon_i} \mathbf{z}$ 
9: return  $\mathbf{x}_0$ 

```

Score-based model with SDE: Sampling

Sampling with Probability Flow ODE:

- For all diffusion process, there exists a corresponding **deterministic process** whose trajectories share the same marginal probability densities. That is:

$$p_t^{SDE}(\mathbf{x}) = p_t^{ODE}(\mathbf{x}) \quad \forall t \in [0, T]$$

This deterministic process satisfies an ODE (called ‘probability flow ODE’), which is given as follows:

$$d\mathbf{x} = \left[f(\mathbf{x}, t) - \frac{1}{2} g(t)^2 \nabla_{\mathbf{x}} \log p_t(\mathbf{x}) \right] dt$$

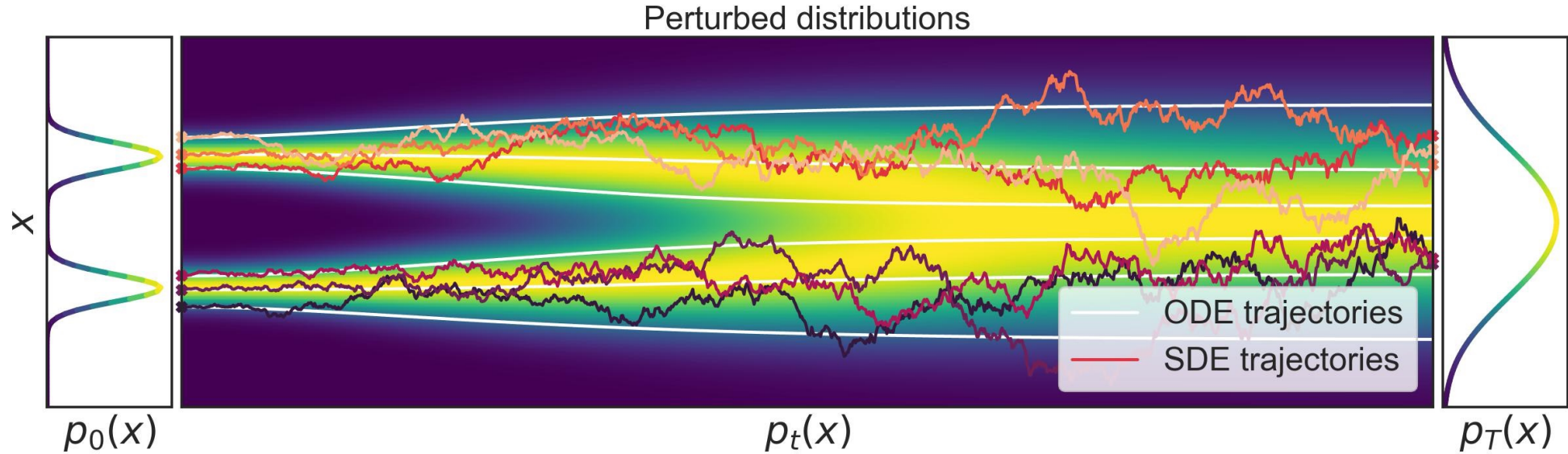
- Since $s_{\theta}(\mathbf{x}, t) \approx \nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, we can sample using the following ODE:

$$d\mathbf{x} = \left[f(\mathbf{x}, t) - \frac{1}{2} g(t)^2 s_{\theta}(\mathbf{x}, t) \right] dt$$

- We can sample $\mathbf{x}(T) \sim \pi$ and solve for $\mathbf{x}(0)$ using an ODE solver.

Score-based model with SDE: Sampling

47



Score-based model with SDE: Experiments

Table 2: NLLs and FIDs (ODE) on CIFAR-10.

Model	NLL Test ↓	FID ↓
RealNVP (Dinh et al., 2016)	3.49	-
iResNet (Behrmann et al., 2019)	3.45	-
Glow (Kingma & Dhariwal, 2018)	3.35	-
MintNet (Song et al., 2019b)	3.32	-
Residual Flow (Chen et al., 2019)	3.28	46.37
FFJORD (Grathwohl et al., 2018)	3.40	-
Flow++ (Ho et al., 2019)	3.29	-
DDPM (L) (Ho et al., 2020)	$\leq 3.70^*$	13.51
DDPM (L_{simple}) (Ho et al., 2020)	$\leq 3.75^*$	3.17
DDPM	3.28	3.37
DDPM cont. (VP)	3.21	3.69
DDPM cont. (sub-VP)	3.05	3.56
DDPM++ cont. (VP)	3.16	3.93
DDPM++ cont. (sub-VP)	3.02	3.16
DDPM++ cont. (deep, VP)	3.13	3.08
DDPM++ cont. (deep, sub-VP)	2.99	2.92

Table 3: CIFAR-10 sample quality.

Model	FID ↓	IS ↑
Conditional		
BigGAN (Brock et al., 2018)	14.73	9.22
StyleGAN2-ADA (Karras et al., 2020a)	2.42	10.14
Unconditional		
StyleGAN2-ADA (Karras et al., 2020a)	2.92	9.83
NCSN (Song & Ermon, 2019)	25.32	$8.87 \pm .12$
NCSNv2 (Song & Ermon, 2020)	10.87	$8.40 \pm .07$
DDPM (Ho et al., 2020)	3.17	$9.46 \pm .11$
DDPM++	2.78	9.64
DDPM++ cont. (VP)	2.55	9.58
DDPM++ cont. (sub-VP)	2.61	9.56
DDPM++ cont. (deep, VP)	2.41	9.68
DDPM++ cont. (deep, sub-VP)	2.41	9.57
NCSN++	2.45	9.73
NCSN++ cont. (VE)	2.38	9.83
NCSN++ cont. (deep, VE)	2.20	9.89

Score-based model with SDE: Experiments



Figure 12: Samples on 1024×1024 CelebA-HQ from a modified NCSN++ model trained with the VE SDE.

Score-based model with SDE: Relation to likelihood learning

- Recall the probability flow ODE:

$$d\mathbf{x} = \tilde{f}_\theta(\mathbf{x}, t)dt$$

where $\tilde{f}_\theta(\mathbf{x}, t) = f(\mathbf{x}, t) - \frac{1}{2}g(t)^2 s_\theta(\mathbf{x}, t)$.

- By the instantaneous change of variable theorem, the log-likelihood of $p_0(\mathbf{x})$ can be computed as:

$$\log p_0(\mathbf{x}(0)) = \log p_T(\mathbf{x}(T)) + \int_0^T \text{tr}(\nabla_{\mathbf{x}(t)} \tilde{f}_\theta(\mathbf{x}(t), t))dt$$

Theorem 1 (Instantaneous Change of Variables). *Let $\mathbf{z}(t)$ be a finite continuous random variable with probability $p(\mathbf{z}(t))$ dependent on time. Let $\frac{d\mathbf{z}}{dt} = f(\mathbf{z}(t), t)$ be a differential equation describing a continuous-in-time transformation of $\mathbf{z}(t)$. Assuming that f is uniformly Lipschitz continuous in $\mathbf{z}$ and continuous in t , then the change in log probability also follows a differential equation,*

$$\frac{\partial \log p(\mathbf{z}(t))}{\partial t} = -\text{tr} \left(\frac{df}{d\mathbf{z}(t)} \right) \quad (8)$$

Score-based model with SDE: Relation to likelihood learning

$$\log p_0(\mathbf{x}(0)) = \log p_T(\mathbf{x}(T)) + \int_0^T \text{tr}(\nabla_{\mathbf{x}(t)} \tilde{f}_\theta(\mathbf{x}(t), t)) dt$$

- The log-likelihood of $p_0(\mathbf{x})$ is computable, so is likelihood learning a great idea?
- Likelihood learning is computationally expensive:
 - Computing $\text{tr}(\nabla_{\mathbf{x}(t)} \tilde{f}_\theta(\mathbf{x}(t), t))$ is costly.
 - Require calling an ODE solver for every optimization step, which is not suitable for large-scale score-based models.
- Fortunately, if we choose the weighting function to be $\lambda(t) = g^2(t)$, then under some regularity conditions:

$$D_{KL}(P || p_\theta) \leq \frac{T}{2} \mathbf{E}_{t \sim U(0, T)} \mathbf{E}_{p_t(\mathbf{x})} [\lambda(t) \|\nabla_{\mathbf{x}} \log p_t(\mathbf{x}) - s_\theta(\mathbf{x}, t)\|_2^2] + D_{KL}(p_T || \pi)$$

- Hence, $\lambda(t) = g^2(t)$ is called the **likelihood weighting function**.

Score-based model with SDE: Pros and cons

■ **Pros:**

- Great image quality.
- Flexible architecture choices for score network, as long as the number of dimensions of input and output are the same.
- Provide exact likelihood computation (for score-based models with SDE).

- **Cons:** require a large number of sampling steps (hundreds or thousands).

- Previous work: using general-purpose ODE solvers for solving the probability flow ODE.
- **DPM-Solver**: dedicated solver for diffusion ODEs.
- Can generate high-quality samples in 10 to 20 steps; 4 – 16 times faster than previous state-of-the-art samplers.

Table 3: Sample quality measured by FID ↓ on CIFAR-10 dataset with continuous-time methods, varying the number of function evaluations (NFE).

Sampling method \ NFE			10	12	15	20	50	200	1000
CIFAR-10 (continuous-time model (VP deep) [3], linear noise schedule)									
SDE	Euler (denoise) [3]	$\epsilon = 10^{-3}$	304.73	278.87	248.13	193.94	66.32	12.27	2.44
		$\epsilon = 10^{-4}$	444.63	427.54	395.95	300.41	101.66	22.98	5.01
	Improved Euler [20]	$\epsilon = 10^{-3}$	82.42(NFE=48), 2.73(NFE=151), 2.44(NFE=180)						
ODE	RK45 Solver [28] [3]	$\epsilon = 10^{-3}$	19.55(NFE=26), 17.81(NFE=38), 3.55(NFE=62)						
		$\epsilon = 10^{-4}$	51.66(NFE=26), 21.54(NFE=38), 12.72(NFE=50), 2.61(NFE=62)						
	DPM-Solver (ours)	$\epsilon = 10^{-3}$	4.70	3.75	3.24	3.99	3.84 (NFE = 42)		
		$\epsilon = 10^{-4}$	6.96	4.93	3.35	2.87	2.59 (NFE = 51)		

Consistency models

- Instead of solving the probability flow ODE, we learn a function that maps a point on the ODE trajectory to its origin.
- Support fast one-step generation, also allow multistep sampling to trade compute for sample quality.

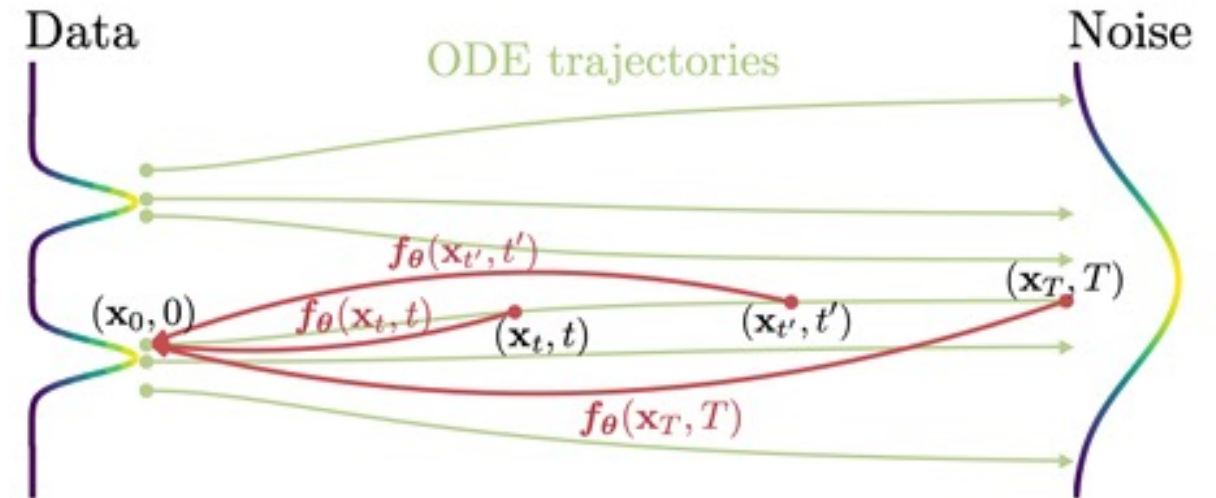


Figure 2: **Consistency models** are trained to map points on any trajectory of the **PF ODE** to the trajectory's origin.

Some useful resources

- [CS236 – Deep Generative Models, Stanford University](#)
- [Yang Song's blog - Generative Modeling by Estimating Gradients of the Data Distribution](#)
- [Training score-based models with SDEs on MNIST dataset using PyTorch](#)